ML E IA: DE DESARROLLO A PRODUCCIÓN · CONTENEDORES 1 DE 2

Contenedores: el modelo mental

Qué es una imagen, qué es un contenedor, por qué se para, qué se pierde al borrarlo y cómo se llega hasta él. Con cuatro prácticas en tu Codespace.

Docker **Imagen y contenedor** **Puertos** **Volúmenes** **Antes de S2**

CÓMO USAR ESTAS DIAPOSITIVAS

Copia cada línea y pégala en la terminal de tu Codespace, **de una en una**, leyendo lo que sale.

Comandos y salidas **reales**: ejecutados con Docker 29.4.2 sobre Ubuntu 26.04.1 LTS (GitHub Actions, 10/11/2026). **Todavía no se han ejecutado en un Codespace del curso**: allí la versión de Docker, las columnas y los tiempos pueden diferir. Los identificadores largos se acortan con «…».

PARA QUÉ SIRVE

«En mi máquina funciona»

LO QUE CAMBIA DE UN EQUIPO A OTRO

- La **versión de Python** y de cada librería.
- El **sistema operativo** y los paquetes que trae.
- Las **variables de entorno** y los archivos que haya por ahí.

LA IDEA

Empaquetar el programa **con todo lo que necesita para ejecutarse** y ejecutar ese paquete donde haga falta: tu Codespace, la integración continua o la nube.

Tu código
la API y el modelo

→

+ Dependencias
las versiones exactas

→

+ Sistema base
un Linux mínimo con Python

→

Imagen
todo empaquetado, de solo lectura

→

Contenedor
una ejecución de la imagen

La misma imagen que pruebas en tu Codespace es la que despliegas en la nube (Cloud Run, en S3): el entorno deja de depender de la máquina donde se ejecuta.

LOS DOS CONCEPTOS

Una imagen es la plantilla; un contenedor, una ejecución

IMAGEN

`python:3.11-slim`

Plantilla de **solo lectura**: un Linux mínimo con Python 3.11. Se descarga una vez y se reutiliza.

→

- Contenedor `hola` — una ejecución
- Contenedor `reloj` — otra ejecución
- Contenedor `web` — otra más

LO QUE DEBES RETENER

- De **una** imagen salen **tantos** contenedores como quieras.
- Una imagen **no se ejecuta**: se usa para crear contenedores.
- Borrar un contenedor **no borra** la imagen.

ASÍ SE VE LA IMAGEN EN TU MÁQUINA (REAL)

```
IMAGE                ID             DISK USAGE
CONTENT SIZE   EXTRA
python:3.11-slim     07d4824036af        189MB
47.8MB
```

docker image ls python:3.11-slim · Docker 29.4.2 · Ubuntu 26.04.1 LTS · GitHub Actions · 10/11/2026

Dentro del contenedor hay otro Linux

MEDIDO EN LAS PRUEBAS

La máquina que ejecutó los ejemplos era **Ubuntu 26.04.1 LTS**. Dentro del contenedor:

```
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
```

Lo que ves es la imagen, no tu máquina. Crear, ejecutar y borrar ese contenedor tardó 0.2 s.

QUÉ ES Y QUÉ NO ES

	MÁQUINA VIRTUAL	CONTENEDOR
Qué trae	Un sistema operativo completo, con su propio núcleo	Los archivos de la imagen y tu proceso
El núcleo (kernel)	El suyo	El de la máquina anfitriona (compartido)
Se parece a…	Otro ordenador dentro del tuyo	Un proceso aislado con su propio sistema de archivos

Tamaño de la imagen base en el equipo de pruebas: 189 MB en disco y 47,8 MB descargados (columnas **DISK USAGE** y **CONTENT SIZE**; según la versión de Docker las columnas pueden variar).

 EJECUTADO CON DOCKER REAL

Práctica A · Imagen y contenedor

Descarga la imagen, mira dentro del contenedor y comprueba que un contenedor parado sigue existiendo hasta que lo borras.

TERMINAL DEL CODESPACE

```
> docker pull python:3.11-slim
> docker image ls python:3.11-slim
> docker run --rm python:3.11-slim sh -c "head -2 /etc/os-release"
> docker run --name hola python:3.11-slim python -c "print('hola desde un contenedor')"
> docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
> docker rm hola
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 1).

```
$ docker image ls python:3.11-slim
IMAGE              ID             DISK USAGE   CONTENT SIZE
EXTRA
python:3.11-slim   07d4824036af        189MB         47.8MB
```

```
$ docker ps -a --format "table
{{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES     IMAGE              STATUS
hola      python:3.11-slim   Exited (0) Less than a second
ago
```

PARA PENSAR · ¿Qué pasa con la imagen cuando borras un contenedor? · ¿Por qué `docker ps` (sin `-a`) no mostraría `hola` aunque exista?

LA REGLA MÁS IMPORTANTE

Un contenedor vive lo que vive su proceso

docker run
crea el contenedor

→

Arranca
su proceso principal

→

El proceso termina
o lo paran

→

Se para
Exited (código)

TERMINA BIEN · `EXITED (0)`

Un `python -c "print(…)"` imprime y acaba.

```
NAMES     IMAGE             STATUS
hola      python:3.11-slim  Exited (0) Less than
a second ago
```

FALLA · `EXITED (2)`

El `CMD` apunta a `aap.py` (errata): Python no lo encuentra (rotura 2).

```
NAMES     STATUS
r2        Exited (2) Less than a second ago
```

LO MATAN · `EXITED (137)`

Un bucle infinito no acaba solo; `docker stop` tardó 10 s y terminó en 137 = 128 + 9 (señal KILL).

```
NAMES     STATUS
reloj     Exited (137) Less than a second ago
```

NO HACE NADA · `EXITED (0)`

`docker run -d python:3.11-slim` sin programa: arranca y acaba al instante (rotura 5).

```
NAMES     STATUS
vacio     Exited (0) 1 second ago
```

LOS CUATRO ESTADOS

Ciclo de vida: crear, arrancar, parar, borrar

Created existe, aún no ha arrancado → **Up** en marcha → **Exited** parado, sigue existiendo → **Borrado** ya no existe

QUIERO…	COMANDO	QUÉ HACE
crear y arrancar	`docker run [-d] [--name x] imagen`	`-d` lo deja en segundo plano
parar	`docker stop x`	pide parar; si no responde, lo mata a los 10 s
volver a arrancar	`docker start x`	conserva su configuración
borrar	`docker rm x` · `docker rm -f x`	`-f` lo para y lo borra
ver estado	`docker ps -a`	sin `-a` solo los que están en marcha
ver registros	`docker logs [--tail n] x`	lo que el proceso escribe en pantalla
entrar a mirar	`docker exec x comando`	ejecuta algo dentro del contenedor en marcha

```
$ docker ps -a --format "table {{.Names}}\t{{.Status}}"
NAMES     STATUS
reloj     Exited (137) Less than a second ago
```

```
$ docker ps --format "table {{.Names}}\t{{.Status}}"
NAMES     STATUS
reloj     Up 1 second
```

 EJECUTADO CON DOCKER REAL

Práctica B · Ciclo de vida

Arranca un contenedor que no termina, mira sus registros, párlo (tardará unos 10 s), comprueba el `Exited (137)` y vuelve a arrancarlo.

TERMINAL DEL CODESPACE

```
> docker run -d --name reloj python:3.11-slim sh -c "while true; do echo tic; sleep 1; done"
> docker logs --tail 3 reloj
> docker stop reloj
> docker ps -a --format "table {{.Names}}\t{{.Status}}"
> docker start reloj
> docker rm -f reloj
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 2).

```
$ docker logs --tail 3 reloj
tic
tic
tic
```

```
$ docker ps -a --format "table {{.Names}}\t{{.Status}}"
NAMES     STATUS
reloj     Exited (137) Less than a second ago
```

```
$ docker ps --format "table {{.Names}}\t{{.Status}}"
NAMES     STATUS
reloj     Up 1 second
```

PARA PENSAR · ¿Qué diferencia hay entre `docker stop` y `docker rm`? · ¿Por qué `docker stop` tardó tanto aquí y una API real suele parar al instante?

EFÍMERO POR DISEÑO

Lo que escribes dentro de un contenedor muere con él

Capas de la imagen `python:3.11-slim` — solo lectura

Tu código y tus archivos de la imagen — solo lectura

Capa de escritura del contenedor — se borra con `docker rm`

Todo lo que el proceso escribe dentro va a la capa de abajo. Si borras el contenedor, desaparece.

```
$ docker run --name efimero python:3.11-slim sh -c "echo dato > /tmp/…
```

```
dato
```

```
$ docker run --rm python:3.11-slim cat /tmp/nota.txt
```

```
cat: /tmp/nota.txt: No such file or directory
```

DÓNDE GUARDAR LO QUE DEBE SOBREVIVIR

- Un **volumen** (siguiente diapositiva).
- Una **carpeta tuya montada** en el contenedor.
- **Fuera**: una base de datos o un almacén de archivos.

DOCUMENTACIÓN · SIN PROBAR AQUÍ Google documenta que el sistema de archivos de un contenedor en Cloud Run está en memoria y que lo escrito no persiste cuando la instancia se detiene (contrato de contenedores, consultado 10/11/2026). Se comprobará en S3.

DATOS QUE SOBREVIVEN

Dos formas de que algo exista fuera del contenedor

VOLUMEN · LO GESTIONA DOCKER

`-v datos:/datos` · sirve para datos que deben sobrevivir (una base de datos, resultados).

```
$ docker run --rm -v datos:/datos python:3.11-slim
cat /datos/nota.txt
```

```
version 1
```

Otro contenedor, distinto, ve el archivo que escribió el anterior.

CARPETA TUYA MONTADA

`-v "$PWD/modelo-demo:/modelos:ro"` · sirve para desarrollo y para cambiar datos o modelo sin reconstruir.

```
$ …cat /modelos/modelo.txt (tras editar el archivo
en tu máquina)
```

```
modelo v2
```

```
$ …sh -c "echo x > /modelos/otro.txt" (con :ro)
```

```
sh: 1: cannot create /modelos/otro.txt: Read-
only file system
```

El proyecto del curso **copia el modelo dentro de la imagen** (`COPY models/…`): la imagen es autosuficiente y reproducible. Montarlo sería más flexible, pero depende de que la carpeta esté donde toca.

 EJECUTADO CON DOCKER REAL

Práctica C · Volúmenes y montajes

Escribe en un volumen desde un contenedor y léelo desde otro; monta una carpeta tuya en solo lectura, cámbiala y comprueba que el contenedor ve el cambio.

TERMINAL DEL CODESPACE

```
> mkdir -p ~/practica-docker && cd ~/practica-docker
> docker volume create datos
> docker run --rm -v datos:/datos python:3.11-slim sh -c "echo version 1 > /datos/nota.txt"
> docker run --rm -v datos:/datos python:3.11-slim cat /datos/nota.txt
> mkdir -p modelo-demo && echo "modelo v1" > modelo-demo/modelo.txt
> docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim cat /modelos/modelo.txt
> echo "modelo v2" > modelo-demo/modelo.txt
> docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim cat /modelos/modelo.txt
> docker run --rm -v "$PWD/modelo-demo:/modelos:ro" python:3.11-slim sh -c "echo x > /modelos/otro.txt"
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 7).

```
$ docker run --rm -v datos:/datos
python:3.11-slim cat /datos/nota.txt
```

```
version 1
```

```
$ docker run --rm -v "$PWD/modelo-
demo:/modelos:ro" python:3.11-slim …
```

```
modelo v2
```

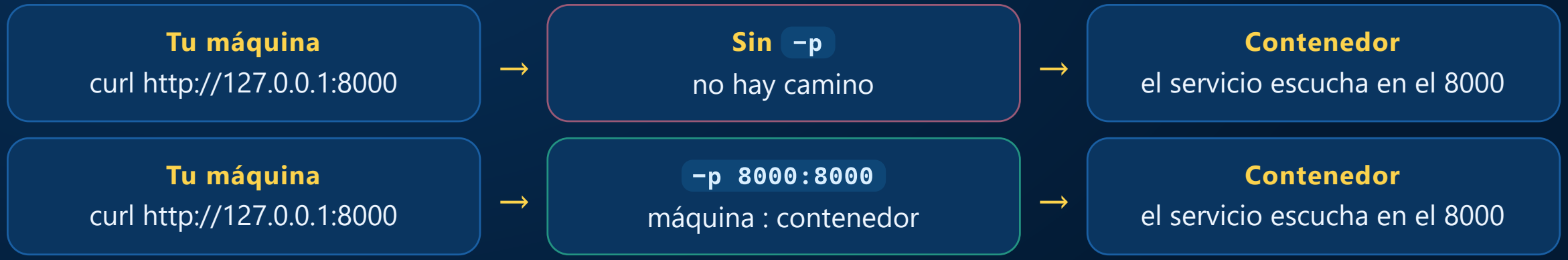
```
$ docker run --rm -v "$PWD/modelo-demo:/modelos:ro"
python:3.11-slim …
```

```
sh: 1: cannot create /modelos/otro.txt: Read-only
file system
```

PARA PENSAR · ¿Qué ventaja y qué riesgo tiene montar el modelo en lugar de copiarlo en la imagen?

LLEGAR AL CONTENEDOR DESDE FUERA

Sin publicar el puerto, no hay camino hasta el servicio




```
$ curl -sS --max-time 3 http://127.0.0.1:8000/
curl: (7) Failed to connect to 127.0.0.1 port
8000 after 0 ms: Could not connect to server
```

```
$ curl -sS -o /dev/null -w "%
{http_code}\n"
http://127.0.0.1:8000/
200
```

```
$ docker port web
8000/tcp →
0.0.0.0:8000
8000/tcp →
[::]:8000
```

A la izquierda de `-p`, el puerto de **tu máquina**; a la derecha, el del **contenedor**. Dos contenedores no pueden publicar el mismo puerto de tu máquina (rotura 1).

LA PREGUNTA DEL LABORATORIO DE S2

Dentro del contenedor, el servicio escucha en 0.0.0.0

ESCUCHA EN `0.0.0.0` (TODAS LAS INTERFACES)

`python -m http.server 8000` con el puerto publicado.

```
$ curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/
200
```

ESCUCHA SOLO EN `127.0.0.1`

El mismo servidor con `--bind 127.0.0.1` y el puerto publicado igual.

```
$ curl -sS --max-time 3 http://127.0.0.1:8000/
curl: (56) Recv failure: Connection reset by peer
```

El tráfico entra por la red del contenedor, no por su «127.0.0.1»: un servicio que solo escucha en el bucle local **no lo atiende nadie de fuera**, aunque el puerto esté publicado. Por eso la API del curso arranca con `--host 0.0.0.0`.

DOCUMENTACIÓN · SIN PROBAR AQUÍ Cloud Run exige lo mismo: el contenedor debe escuchar en `0.0.0.0` en el puerto al que se envían las peticiones (8080 por defecto) y no en `127.0.0.1` (contrato de contenedores, 10/11/2026).

 EJECUTADO CON DOCKER REAL

Práctica D · Puertos

Compruébalo tú: primero sin publicar el puerto, luego publicándolo, y por último con el servicio escuchando solo en 127.0.0.1. Espera 2 s tras cada `docker run -d` antes del `curl`.

TERMINAL DEL CODESPACE

```
> docker run -d --name web python:3.11-slim python -m http.server 8000
> curl -sS --max-time 3 http://127.0.0.1:8000/
> docker rm -f web
> docker run -d --name web -p 8000:8000 python:3.11-slim python -m http.server 8000
> curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/
> docker rm -f web
> docker run -d --name web -p 8000:8000 python:3.11-slim python -m http.server 8000 --bind 127.0.0.1
> curl -sS --max-time 3 http://127.0.0.1:8000/
> docker rm -f web
```

Selección de comandos; el ejemplo completo está en la guía práctica (ejemplo 3).

```
$ curl -sS --max-time 3 http://127.0.0.1:8000/
curl: (7) Failed to connect to 127.0.0.1 port 8000 after 0 ms:
Could not connect to server
```

```
$ curl -sS -o /dev/null -w "%
{http_code}\n" http://127.0.0.1:8000/
200
```

```
$ curl -sS --max-time 3
http://127.0.0.1:8000/
curl: (56) Recv failure: Connection
reset by peer
```

PARA PENSAR · La respuesta a la pregunta de S2: ¿por qué el contenedor de la API escucha en `0.0.0.0` y no en `127.0.0.1`?

PARA LLEVARTE

Seis ideas que ya puedes usar

1. IMAGEN ≠ CONTENEDOR

La imagen es la plantilla de solo lectura; el contenedor, una ejecución de ella.

2. VIVE LO QUE VIVE SU PROCESO

Si el proceso termina, el contenedor se para. `docker ps -a` + `docker logs` dicen por qué.

3. LO ESCRITO DENTRO SE PIERDE

Para que algo sobreviva: volumen, carpeta montada o almacenamiento externo.

4. HAY QUE PUBLICAR EL PUERTO

`-p máquina:contenedor`. Un puerto de tu máquina, un contenedor.

5. ESCUCHA EN 0.0.0.0

En `127.0.0.1` nadie de fuera llega, aunque publiques el puerto.

6. DENTRO HAY OTRO LINUX

Lo que ves es la imagen, no tu máquina; el núcleo sí es el del anfitrión.

Listo para el Dockerfile

- [ ] He descargado una imagen y he ejecutado un contenedor con `--rm` y otro sin él. — Práctica A
- [ ] Sé leer `docker ps -a` y explicar `Exited (0)` y `Exited (137)`. — Práctica B
- [ ] He comprobado que lo escrito dentro de un contenedor desaparece al borrarlo. — Ejemplo 2
- [ ] He usado un volumen y una carpeta montada en solo lectura. — Práctica C
- [ ] He publicado un puerto con `-p` y sé por qué el servicio debe escuchar en `0.0.0.0`. — Práctica D

Comprueba cada punto antes de pasar a lo siguiente.

Siguiente: Del Dockerfile a producción →